Prolog

· Laborator 1

Atomi: elefant, ' (@ •+' , + , abcXYZ

Variabile: _x , X_1_2, Animal , X

Constante
- atomi
- numere

T = U (reușește dc. ∃ o potrivire - termenii se unifică)

T \= U (reușește dc. ∄ o potrivire)

T == U (reușește dc. termenii SUNT identici)

T \== U (reușește dc. termenii SUNT diferiți)

2 =:= 1+1 → true

\+ animal (cat) / not (animal (cat))

· Laborator 2

?- 3+5 = + (3,5). → true

?- 3+5 = + (5,3). → false

?- 3+5 = 8 → false

?- X = 4, 8 is 3+X → false

?- X = 4, 7 is 3+X → X = 4

?- 3+5 is 8. → false

?- X is 3+5.

X = 8

?- 8 is 3+X

is/2: Arguments are not sufficiently instantiated

Scanat cu CamScanner

?- x is 9/4	?- x is 30-4	?- x is 9/4
x = 2.25	x = 26	x = 2.25

=:= / =\= → diferit

e recomandat =:= în loc de is

?- 2 ** 3 =:= 3+5.	?- 2 ** 3 = 3+5
true	false

Fibonacci: 1 1 2 3 5 8 13 21 34

fibo(M, Z, X)

X = fibonacci(M) → val. pe care o vrem

Z = — " — (M-1) → val ant

Y = — " — (M-2) → encă una, pt. recursie

fibo(4, Z, X)

$\boxed{M = 3}$

fibo(3, Y, Z) / X = Y + Z

---

nl → linie goală

write → pt afișare

8 is 3+5 ~~false~~ tre

3+5 is 8 false

Scanat cu CamScanner

UTIL

length ([a, b, c, d], x).

X = 4

member (b, [a, b, c, d]).

true

append ([1,2,3,4], [a,b,c], REZ).

REZ = [1, 2, 3, 4, a, b, c]

last ([i, u, l, i, a], a).

true

last ([i, u, l, i, a], i).

false

reverse ([i, u, l, i, a], [a, i, l, u, i])

true

reverse ([l, u, l, i, a], [a, i, l, u, i])

false

Scanat cu CamScanner

Liste

?- [1, 2, 3, 4, 5] = [Head | Tail]

Head = 1

Tail = [2, 3, 4, 5]

? [quod, licet, jovi, non, licet, bovi] = [_, x | _]

x = licet

al 2 lea elem. dintr-o listă

```
element_of (x, [x|_]).
element_of(x, [_|Tail]) :- element_of (x, Tail).
```

```
concat_lists ( [], List, List).
concat_lists ( [Elem | List1], List2 , [Elem | List3]) :-
concat_lists (List1, List2, List3).
```

concat_lists ([1, 2, 3], [d, e, f, g], x).

x = [1, 2, 3, d, e, g, h]

Scanat cu CamScanner

Laborator 4

bagof/3 / setof/3 / findall/3 → nu mai avem există (^)

setof/3 → elimină duplicatele

① bagof/3 | bagof(Rezultat, Condiție, Listă) → colectează Rezultat care satisf Condiție și le pune în Listă

→ adună toate sol. posibile, păstrează repetițiile, dar poate produce grupări dc. ∃ var. libere.

bagof(M, listaNelem(L, N, M), LL)

||

găsește toate valorile pos. pt. M pt. care e adev. listaNelem(L, N, M) și pune-le într-o listă LL.

Dc. în Condiție apar variabile libere (i.e. nelegate, care nu apar în Rezultat), bagof/3 grupează rez. separat pt. fiecare val. posibilă a acelei variabile.

Scanat cu CamScanner